Ders 1 Genişletilmiş Notu: Derse Giriş: Ders İçeriği, Grup Projesi ve Yazılım Mühendisliğinde Kalitenin Yeri

Ders 1 Genişletilmiş Notu: Derse Giriş: Ders İçeriği, Grup Projesi ve Yazılım Mühendisliğinde Kalitenin Yeri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yazılım Mühendisliği İçindeki Yeri
- 2. Dersin Bölümleri
- 3. Sektörde Yazılım Kalitesi ve Test Birimleri
- 4. Grup Projesinin Teknik İçeriği
- 5. Gereksinim Tipleri
- 6. Yazılım Mühendisliğinin Tanımı ve Katmanlı Yapısı
- 7. Süreç Adımları ve Destek Süreçleri
- 8. Yazılım Geliştirme Süreç Modelleri
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Yazılım Mühendisliği İçindeki Yeri

Bu ilk derste hoca ekran paylaşımı yapmadan, dersin içeriğini ve yazılım mühendisliğiyle bağlantısını anlattı. Dersin ana konusu, yazılım geliştirme sürecinde kalite yaklaşımıdır. Bu konuya yazılım mühendisliği dersinde başlanmıştı.

Hoca, yazılım mühendisliği dersini üç ya da dört bölümde incelediklerini hatırlattı:

- **Genel kavramlar:** Yazılım mühendisliğinin ne olduğu, hangi alanlarda kullanıldığı ve yazılım geliştirme süreç modelleri.
- **Süreç adımları:** Sistem analizi dersinden farklı adlarla da bilinen analiz, tasarım, gerçekleştirme ve test adımları. Yazılım mühendisliği dersinde bu adımlar nesneye yönelik yaklaşımla ayrıntılı işlenmişti.
- **Destek yaklaşımları ve aktiviteler:** Bunların içinde iki konu vardı. Biri yazılım proje yönetimi, öbürü yazılım kalite yönetimi ve test adımının daha ayrıntılı ele alınmasıydı.

Bu ders, son maddedeki kalite yaklaşımını ve test süreçlerini ayrıntılı işler. Bu nedenle ders, yazılım mühendisliği dersini tamamlamış öğrencilere yöneliktir.

2. Dersin Bölümleri

Hoca dersi yaklaşık sekiz alt bölüme ayırdı. Dersin içindekiler listesini içeren slaytı sisteme yükleyeceğini, ama derslerde slayt dışında da konular açacağını söyledi.

2.1 Birinci Bölüm: Süreç Modelleri ve Kalitenin Önemi

Birinci bölümde yazılım geliştirme süreç modellerinin üzerinden özet olarak tekrar geçilecektir. Özellikle spiral model ve çevik (agile) yaklaşımlar ele alınacak, çevik yaklaşımlarda test sürecinin nasıl ilerlediğine bakılacaktır. Burada test güdümlü yazılım geliştirme (Test Driven Development, TDD) sürecinin çevik projelerde nasıl yürütüldüğü de işlenecektir. Bölümün asıl sorusu şudur: Yazılım geliştirme sürecinde kalite yaklaşımı derken ne anlıyoruz?

2.2 İkinci Bölüm: Yazılım Kalitesinin Temel Kavramları

Teste geçmeden önce kaliteyle ilgili temel kavramlar görülecektir. Kalite özellikleri yazılım mühendisliği dersinde de çok üzerinde durulan bir konuydu. Hoca, McCall'a göre kalite etkenlerinin ve kalite özelliklerinin neler olduğunu sınavlarda da sorduğunu söyledi.

> **Hocanın vurgusu:** McCall'a göre kalite etkenleri (faktörleri) ve kalite özellikleri sınavlarda sorulmuştur. Kalite faktörleri ile yazılım özellikleri arasındaki ilişkiyi gösteren McCall matrisi bu derste yeniden ele alınacaktır.
>
> **Ek bilgi (derste anlatılmadı):** McCall kalite modeli, büyük olasılıkla 1977'de önerilmiş, yazılım kalitesini kullanıcı bakışından "faktörler"e, geliştirici bakışından "kriterler"e ayıran ilk modellerdendir. Faktörler ürünün işletimi, değiştirilmesi ve başka ortama aktarılması başlıkları altında toplanır. Faktörler ile kriterler arasındaki ilişki bir matrisle gösterilir; hocanın "matris" dediği yapı budur.

Bu bölümün dersin kendine özgü kısmı, yazılım kalitesinin sağlanması için hangi aktivitelerin yapıldığıdır. Bunlar arasında yazılım geliştirme sürecindeki gözden geçirmeler (review), yazılım kalite standartları ve yazılım inceleme aşaması sayıldı.

2.3 Üçüncü Bölüm: Geçerleme ve Doğrulama

Üçüncü bölümde geçerleme ve doğrulama (verification, validation) kavramları açılacaktır. Kalite yönetim sistemi derken ne anlatıldığı ve kalite yönetiminde geçerleme ile doğrulamanın yeri görülecektir. Gözden geçirme, onaylama ve inceleme ayrıntılandırılacaktır.

Bu bölümde şu soru da irdelenecektir: Test, yazılım geliştirme sürecinde nerede yer alır? "Test en sonunda yapılır" düşüncesi doğru mudur? Kullanılan geliştirme sürecine bağlı olarak test ya en sonda yapılır ya da TDD'de olduğu gibi geliştirmeyle birlikte yürür. Farklı süreç modelleri içinde bu durum farklı alt başlıklar olarak karşımıza çıkacaktır. Kaliteyi sağlamak için ikinci bölümde sayılan aktivitelere burada geçerleme ve doğrulamayla başlanmış olacaktır.

> **Ek bilgi (derste anlatılmadı):** Yaygın kullanılan ayrıma göre doğrulama (verification) "ürünü doğru mu yapıyoruz?" sorusunu, yani ürünün belirtimlere uygunluğunu sorar. Geçerleme (validation) ise "doğru ürünü mü yapıyoruz?" sorusunu, yani ürünün kullanıcının gerçek ihtiyacını karşılayıp karşılamadığını sorar. Türkçe kaynaklarda iki terimin karşılıkları zaman zaman yer değiştirerek kullanılır.

2.4 Dördüncü Bölüm: İnceleme (Inspection)

Dördüncü bölüm yazılım kalite yönetimi içinde incelemeye (inspection), yani geliştirilen yazılımın incelenmesi aşamasına ayrılmıştır. Hoca incelemeyi kalitenin sağlanması için yapılan önemli aktivitelerden biri olarak niteledi. Ele alınacak sorular şunlardır:

- Gözden geçirme ile inceleme arasındaki fark nedir?
- Kod inceleme denince ne anlaşılır?
- Fagan'ın önerdiği inceleme süreci nasıl yürür?
- İnceleme ekibi nasıl oluşturulur?
- Teknik inceleme nasıl yapılır?
- İnceleme aşaması profesyonel hayatta, yazılım sektöründe nasıl işlenir?

> **Ek bilgi (derste anlatılmadı):** Fagan incelemesi, büyük olasılıkla 1970'lerde IBM'de Michael Fagan tarafından tanımlanmış biçimsel bir inceleme yöntemidir. Belirli rolleri (moderatör, yazar, okuyucu, denetçi gibi) olan bir ekip, planlama, toplantı, düzeltme ve izleme gibi aşamalardan geçerek belge ya da koddaki hataları

arar. Amaç hatayı test aşamasından önce, düzeltmenin daha ucuz olduğu bir noktada bulmaktır.

2.5 Beşinci Bölüm: Test Adımları ve Test Teknikleri

Beşinci bölümde test adımlarına geçilecektir. Test süreci yine geçerleme ve doğrulamayla birleştirilerek ele alınacaktır. Bu konular yazılım mühendisliği dersinde özet olarak görülmüştü. Hoca bu bölümün oldukça kapsamlı olacağını söyledi.

- Test süreci adımları: birim testi (unit test), bütünlük testi (integration test), regresyon testi (regression test), sistem testi, onaylama testi (acceptance test).
- Test tipleri: kara kutu testi (black box), gri kutu testi (grey box), saydam kutu testi (white box).

2.6 Altıncı Bölüm: Yazılım Kalite Ölçümü ve Güvenirlik

Hocaya göre kalite ile ölçme birbirini tamamlar. Yazılım kalitesini sağlamak için ölçme yapmamız gerekir. Hoca ölçmenin üç gerekçesini saydı:

- Bitirdiğimiz bir yazılımı değerlendirmek için ölçeriz.
- Başlayacağımız bir yazılımın değerini tahmin edebilmek için ölçeriz.
- Kaynaklarımızı yönetmek için ölçeriz.

Bu bölümün konusu yazılım ölçümüdür (software metrics). Kalite ölçütlerinin neler olduğu ve kalitenin hangi metriklerle ölçüldüğü görülecektir. Hoca, bir şeyin ölçülmesi ile ölçüm sistemleri arasındaki ayrımın kalite ölçümünde de geçerli olduğunu söyledi. Metrikler arasında Halstead metrikleri ve özellikle iç testte (saydam kutu testinde) kullanılan McCabe metriği anılacaktır. Ölçmeye ikinci bölümde kalite özellikleriyle bir ölçüde girilmiş olacak; altıncı bölümde ölçüm metriklerinin kalite özellikleriyle ilişkisi de kurulacaktır.

> **Ek bilgi (derste anlatılmadı):** Halstead metrikleri, programdaki farklı ve toplam operatör ile operand sayılarından program uzunluğu, hacim ve zorluk gibi büyüklükler türetir. McCabe'in çevrimsel karmaşıklığı (cyclomatic complexity) ise programın kontrol akış grafiğindeki bağımsız yol sayısını verir; bu nedenle saydam kutu testinde gereken en az test yolu sayısını belirlemekte kullanılır.

Bu bölümde yazılım güvenirliğinden (reliability) de söz edilecektir: güvenirliğin nasıl ölçüldüğü, güvenirlik modellerinin neler olduğu ve güvenirliğin nasıl sağlandığı. Hoca bu konuyu altıncı bölümün içine koyduğunu, ama ayrı bir bölüm de sayılabileceğini söyledi. Güvenirliğin altıncı bölümde yer almasının nedeni, hem bir ölçüm kriteri hem de bir kalite özelliği olmasıdır.

2.7 Yedinci Bölüm: Test Planlama ve Yönetim

Beşinci ve altıncı bölümlerde test süreci, test teknikleri ve metrikler görülmüş olacaktır. Yedinci bölümün hedefi, yazılım geliştirme aşamasında kalite yönetiminin ve test yönetiminin nasıl gerçekleştirildiğidir:

- Test ekibi nasıl yönetilir?
- Test aktiviteleri nasıl yönetilir?
- Test planı nedir?
- Test senaryosu nasıl hazırlanır?
- Test gerçekleştirilirken sonuç raporu nasıl yazılır?

Bu bölümde teslim sonrasında kalitenin nasıl sağlandığı da işlenecektir: yazılım bakımı, yazılım konfigürasyonu ve bakım maliyetlerinin nasıl azaltılacağı. Hoca bunları da test planlama ve test yönetimi kapsamında saydı.

2.8 Sekizinci Bölüm: Kalite İyileştirme ve CMMI

Yedinci bölüme kadar kalitenin ne olduğu ve testin nasıl yapıldığı anlaşılmış olacaktır. Son bölümün sorusu şudur: Kaliteyi iyileştirmek için neler yapılır? Burada yazılım mühendisliği dersinde adı geçen CMMI ele alınacaktır. CMMI düzeyleri (hoca "adımlar" yerine "düzeyler"

demenin daha doğru olduğunu belirtti), anahtar süreç alanları (key process areas) ve People CMM gibi CMMI'a katılan yapılar işlenecektir.

2.9 Dersin Kapsamı Üzerine Bir Soru

Hoca öğrencilere, dersten başka bir dersin adıyla bekledikleri bir konu olup olmadığını sordu. Bu sırada tasarım kalıpları konusunda bir açıklama yaptı:

Soru: Tasarım kalıpları (design patterns) bu dersin konusu mu? **Cevap:** Hayır. Tasarım kalıpları nesneye yönelik tasarım dersinin ve yazılım mühendisliği dersinin konusudur. Kalite dersi tasarım kalıplarıyla doğrudan ilişkili değildir.

3. Sektörde Yazılım Kalitesi ve Test Birimleri

Bir öğrenci, iş ilanlarında "kalite mühendisliği" diye bir pozisyon gördüğünü belirtti. Hoca bunu, ders içeriğini anlatırken de değindiği bir gelişmeyle bağladı.

Soru: İş ilanlarında kalite mühendisliği pozisyonu geçiyor; bu ders bununla ilişkili mi? **Cevap:** Evet. Yazılım sektöründe artık, başka ürünlerin kalite yönetim birimleri gibi, yazılım kalite birimleri ve test birimleri önem kazanmıştır. Birçok büyük firmanın yazılım biriminde ya da yazılım evlerinde kalite birimi veya test birimi ayrı ekipler olarak bulunur. Bu dersteki kalite özellikleri, kaliteyi gerçekleştirmek için yapılan aktiviteler ve kalite ölçüm sistemleri, sektörde yazılım kalitesi üzerinde çalışanların bilmesi gereken konulardır. Sayıları çok olmasa da, başka firmaların geliştirdiği yazılımlar üzerinde yalnızca test aktivitesi yürüten işler de vardır.

Hoca ayrıca bazı kuruluşlarda sistem analizini yapan ekip ile kullanıcı testini yapan ekibin zaman zaman birleştirildiğini söyledi.

Hoca bu dersin bölümde uzun süredir seçmeli ders olarak verildiğini, dersi yüksek lisanstaki birikimiyle açtığını anlattı. Derse uygulamadaki yaklaşımlar açısından sektörden katkı da alınmıştır. TÜBİTAK Marmara'da güçlü bir kalite grubu olduğunu, bu grubun daha önceki yıllarda ASELSAN'da geliştirilen bazı uygulamaların test işlerini üstlendiğini söyledi. Hoca, uzaktan eğitimde de en az bir derse sektörden konuk davet etmeyi planladığını belirtti. Konuktan beklenen; kalite ve test ekiplerinin yaptığı işleri, test senaryolarının nasıl oluşturulduğunu ve test süreci adımlarının nasıl yönetildiğini anlatmasıdır.

4. Grup Projesinin Teknik İçeriği

Dersin bir grup projesi vardır. Hoca bu işin gerçek hayatta da ekiplerle yapıldığını, gerçek hayatta test görevlerinin ekip içinde paylaşıldığını vurguladı.

4.1 Yazılım Mühendisliği Projesinden Farkı

Yazılım mühendisliği projesinde birkaç konu verilmiş, ekip bunlardan birini seçip yazılım geliştirmişti. Orada tasarıma önem verilmiş, geliştirme sonunda testin bir bölümü yer almıştı. Gereksinim analizi de istenmişti: tasarım öncesinde kullanım durumları (use case) ve kullanım durumu belgesiyle, bazı gruplarda ayrıca bir gereksinim listesiyle.

Bu derste ise yeni bir yazılım geliştirilmez. Var olan bir yazılım test edilir.

4.2 Projenin Adımları

- **Yazılımın seçilmesi:** Test edilecek yazılım grup üyelerinden birinin önceki projesi, başka bir derste üretilmiş bir proje ya da seçilen başka bir yazılım olabilir. Çok küçük bir yazılım seçilmemelidir.
- **Gereksinim listesi ve gereksinim raporu:** Yazılımın her özelliği madde madde yazılır. Liste 50-70 arası gereksinim içermelidir.
- **Kullanıcı testi (zorunlu):** Gereksinim listesine dayanarak yapılır. Bu yüzden gereksinim analizinde görülen gereksinim listesinin çıkarılması gerekir.
- **Birim testi ya da sistem testi (grubun seçimine göre):** Birim testi için yazılımın kaynak kodunun elde olması gerekir. Sistem testi ise dayanıklılığın test edilmesi, sistemle uyumun test edilmesi gibi test bölümünde görülecek sistem testi tiplerini kapsar.

- **Test planı:** Yedinci bölümde görülecek test yönetimi bilgisinin projede uygulanması beklenir.
- **Test raporları ve sonuç raporu.**
- **Sunum:** Gereksinimden sorumlu üye önce yazılımı tanıtır ve gereksinimleri verir. Ardından grup test ekibini tanımlar ve yaptığı testleri açıklar. Böylece bir test işlemi baştan sona bütün olarak gerçekleştirilmiş olur.

Grup büyüdükçe yazılımın kapsamı da genişletilmeli, yapılan test sayısı artırılmalıdır.

5. Gereksinim Tipleri

Hoca, projede gereksinim listesi isteneceği için yazılım mühendisliğinden gereksinim tiplerini hatırlattı. Gereksinim (requirement) yerine "ister" de denir.

5.1 İşlevsel Gereksinimler

İşlevsel gereksinimler (functional requirements), yazılımın gerçekleştirdiği işle ilişkilidir; yazılımın hangi işleri yapacağını tanımlar. Hocanın örneği bir öğrenci bilgi sistemiydi: Öğrencinin harf notunu doğru hesaplaması ve transkripti uygun biçimde vermesi işlevsel gereksinimlerdir.

5.2 İşlevsel Olmayan Gereksinimler

İşlevsel olmayan gereksinimler (non-functional requirements), yazılımın yaptığı özel işle ilişkili değildir; bütün yazılımlar için geçerli genel özelliklerdir. Hoca bunların kalite özellikleriyle eşleştirilebileceğini söyledi. Örnekler:

- Güvenirliğin (reliability) yüksek olması, yani verdiği sonuçların güvenilir olması.
- Kullanıcı dostu olması.
- Güvenlikli olması, güvenliğinin (security) yüksek olması.
- Hızlı çalışması, hızlı sonuç vermesi.

İşlevsel olmayan gereksinimler bütün yazılımlar için sayılabilir, ama bazı yazılımlarda bazıları öne çıkar. Örneğin bir yazılımda güvenlik, bir başkasında hızlı yanıt çok önemlidir.

6. Yazılım Mühendisliğinin Tanımı ve Katmanlı Yapısı

6.1 Hocanın Kısa Tanımı

Hoca, yazılım mühendisliği için kitaptakinden daha kısa ama ilgili bütün gereksinimleri ortaya koyan bir tanımı olduğunu hatırlattı:

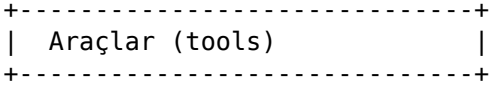
> Yazılım mühendisliği; bilgisayar bilimi, yönetim bilimi, ekonomi ve bilişim dallarından yararlanan, sorun çözümünde mühendislik yöntemlerini kullanan ve kaliteli yazılımı en ekonomik biçimde elde etmek üzere mühendislik yöntemlerini kullanan bilim dalıdır.

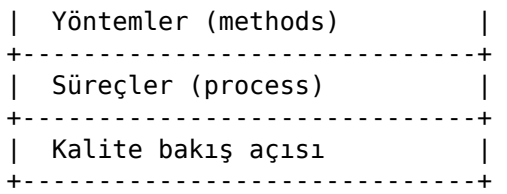
Hoca bu tanımın parçalarını tek tek yorumladı:

- "Kaliteli yazılım" ifadesi tanımın içine kalite yönetimini koyar.
- "En ekonomik biçimde" ifadesi proje yönetimini koyar.
- "Mühendislik yöntemleri" ölçme ve tasarımı içerir.
- "Sorun çözümü" ifadesi yazılımın tanımıyla bağlanır: Yazılım, bir problemin bilgisayara aktarılması ve çözülmesi için gerçekleştirilen uygulamalardır.

6.2 Katmanlı Yapı

Yazılım mühendisliğinin temelinde bir kalite bakış açısı vardır. Yazılım mühendisliği dersinde çizilen katmanlı şekil, alttan üste şöyledir:





Araçlar, alttaki katmanlara uygun olarak seçilir. Hangi aracın kullanılacağına karar verilirken hangi yöntemlerin kullanıldığına, hangi süreçlerden geçildiğine ve kurumun kalite bakış açısının ne olduğuna bakılır. Yazılım mühendisliği bunlara bağlı olarak gerçekleştirilir.

7. Süreç Adımları ve Destek Süreçleri

7.1 Temel Süreç Adımları

Yazılım geliştirme sürecinin temel adımları gereksinim analizi, tasarım, gerçekleştirme (kodlama) ve testtir. Bunların ardından bakım süreci gelir. Hoca, bütün bu geliştirme sürecinin aynı zamanda kalite yaklaşımının gerekli adımları olduğunu belirtti.

7.2 Destek Süreçleri (Umbrella Activities)

Temel adımların yanında, bütün süreci kapsayan destek süreç adımları (umbrella activities) vardır:

- Proje yönetimi
- Kalite yönetimi
- Geçerleme ve doğrulama (validation, verification)
- Formal teknik gözden geçirme
- Konfigürasyon yönetimi
- Yeniden kullanılabilirlik (reusability) yönetimi
- Yazılım ölçümü
- Risk yönetimi

Bunlardan bu dersle ilişkili olanlar kalite yönetimi, geçerleme ve doğrulama, formal teknik gözden geçirme, konfigürasyon yönetimi, yazılım ölçümü ve risk yönetimidir. Hoca bunları dersin bölümleriyle şöyle eşleştirdi:

- Geçerleme ve doğrulama, gözden geçirme ve inceleme dersin üçüncü ve dördüncü bölümlerindedir.
- Yazılım konfigürasyon yönetiminin üzerinden son bölümlerde geçilecektir.
- Risk yönetimi yazılım mühendisliğinde görülmüştür ve daha çok proje yönetimiyle ilişkilidir. Ancak risklerin ne kadar kontrol edilebildiği yazılımın kalitesini etkiler.
- Yeniden kullanılabilirlik dersin alt başlıkları arasında yoktur, ama bir kalite özelliği olarak ele alınacaktır. Hoca, nesneye yönelik yaklaşımın, özellikle tasarım kalıpları kullanılarak, yeniden kullanılabilirliği artırdığını hatırlattı.

8. Yazılım Geliştirme Süreç Modelleri

Hoca süreç modellerinin bir sonraki derste özetleneceğini söyledi ve bunları kısaca saydı.

- **Klasik süreç modelleri:** şelale (waterfall), prototip model, artımlı (incremental) model, spiral model, hızlı uygulama geliştirme (Rapid Application Development, RAD), birleşik süreç (Unified Process) modeli.
- **Çevik modeller:** Scrum, Kanban ve XP (Extreme Programming). Başka çevik modeller de vardır, ama yazılım mühendisliği dersinde en çok bu üçü üzerinde durulmuştu.

Bu modellerin her birinde test aşamasına ve kalite yönetimine yaklaşım farklı olabilir. Kalite ve test için temel bir hedef vardır, ama bu hedefin hangi modelde nasıl sağlandığı farklılık gösterir. Bir sonraki derste spiral modelin ve şelale modelinin üzerinden yalnızca hatırlatma amacıyla hızlıca geçilecektir.

Örnek Sorular

1. (derste sözlü, 00:44:16) Gereksinim (requirement) dediğimizde hangi tip gereksinimlerden bahsediyorduk?

Derste verilen cevap: İki tip vardır. İşlevsel gereksinimler (functional) yazılımın yaptığı işi tanımlar; örneğin bir öğrenci bilgi sisteminin harf notunu doğru hesaplaması. İşlevsel olmayan gereksinimler (non-functional) işe bağlı olmayan genel özelliklerdir; güvenirlik, kullanıcı dostu olma, güvenlik ve hızlı yanıt bunlara örnektir.

2. (derste sözlü, 00:52:30) Hocanın yazılım mühendisliği için yaptığı kısa tanım nedir?

Derste verilen cevap: Yazılım mühendisliği; bilgisayar bilimi, yönetim bilimi, ekonomi ve bilişim dallarından yararlanan, sorun çözümünde mühendislik yöntemlerini kullanan ve kaliteli yazılımı en ekonomik biçimde elde etmek üzere mühendislik yöntemlerini kullanan bilim dalıdır.

3. (derste sözlü, 00:56:14) Yazılım geliştirme sürecindeki süreç adımlarını nasıl saymıştık?

Derste verilen cevap: Gereksinim analizi, tasarım, kodlama (gerçekleştirme) ve test; ardından bakım.

4. (derste sözlü, 01:02:13) Çevik modeller için örnek verebilir misiniz?

Derste verilen cevap: Scrum, Kanban ve XP.

Dersin Toparlaması

Bu derste Yazılım Kalite ve Test Süreci dersinin yazılım mühendisliğinin devamı olduğu görüldü. Yazılım mühendisliğinin tanımında ve katmanlı yapısında kalite bakış açısı en alttaki temeldir. Kalite yönetimi, geçerleme ve doğrulama, formal teknik gözden geçirme, konfigürasyon yönetimi, yazılım ölçümü ve risk yönetimi gibi destek süreçleri bu dersin ana konularını oluşturur. Ders sekiz bölüm halinde süreç modellerinden kalite kavramlarına, geçerleme ve doğrulamadan incelemeye, test adımlarından ölçüme, test yönetiminden CMMI ile kalite iyileştirmeye ilerleyecektir. Grup projesinde var olan bir yazılımın gereksinim listesi çıkarılacak, buna dayalı kullanıcı testi ile birim ya da sistem testi yapılacak, test planı ve raporları hazırlanacaktır.

Bir sonraki derste yazılım geliştirme süreç modelleri (özellikle şelale ve spiral model) hızlıca hatırlatılacaktır. Hoca hazırlık olarak yazılım mühendisliği notlarındaki kaliteyle ilgili bölümlerin incelenmesini istedi.

> **Ders sonrası not:** Altyazıdaki bazı adlar konuşma tanıma sırasında bozulmuştur. Doğruları: “yazılım kayıta” yazılım kalite; “Makkol/Makkola” McCall; “Makkep” McCabe; “hashtag metriği” büyük olasılıkla Halstead metrikleri; “Software Matrix” software metrics; “Spray/Sprower model” spiral model; “ecail” agile; “test driving development” test driven development; “Reizability/J-Yuzubility” reusability; “PeopleCM” People CMM; “Presma” Pressman; “Summerville” Sommerville.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.